PLAN DE DEVELOPPEMENT DE APP.PY **3**
ÉTAPE 1 : DEFINIR LA MISSION DU FICHIER 3
Rôle général de app.py *3*
Remarque importante 3
ÉTAPE 2 : IMPORTER LES OBJETS ET LES CONSTANTES GLOBALES UTILES 3
Code étudié *3*
import os *3*
from http.server import HTTPServer, BaseHTTPRequestHandler *4*
HTTPServer 4
BaseHTTPRequestHandler 4
from urllib.parse import urlparse, parse_qs *4*
urlparse 4
parses 5
from config import APP_HOST, APP_PORT *5*
from routes import ROUTES *5*
STATIC_DIR = os.path.join(os.path.dirname(__file__), "static") *6*
CONTENT_TYPES = {"css": "text/css", "js": "application/javascript"} *6*
Résumé de l'étape *6*
Ce qu'il faut retenir 6
ÉTAPE 3 : CREER LA CLASSE REQUESTHANDLER 7
Rôle de la classe RequestHandler *7*
do_GET(self) *7*
Déclaration de la classe *7*
Pourquoi cette écriture est importante *8*
RequestHandler 8
BaseHTTPRequestHandler 8
Ce que cette classe va contenir 8
Ce qu'il faut retenir 8
Résumé simple 8
do_POST(self) *9*
Rôle de do_GET(self) 9
Code 9
Logigramme 10
Explication ligne par ligne *11*
parsed = urlparse(self.path) 11
path = parsed.path 11
if path.startswith("/static/"): 11
self._serve_static(path) 11
return 11
route = ROUTES.get(("GET", path)) 11
if route is not None: 11
route(self) 12
else: 12
self._send_response(404, "<h1>404 - Page non disponible</h1>") 12
Ce qu'il faut retenir *12*
Résumé simple 12
do_POST(self) *13*
Pourquoi ce nom est important 13
Rôle de do_POST(self) 13
Code 13
Logigramme 14
Explication ligne par ligne *15*
parsed = urlparse(self.path) 15
path = parsed.path 15
route = ROUTES.get(("POST", path)) 15
if route is not None: 15
route(self) 15
else: 15
self._send_response(404, "<h1>404 - Page non disponible</h1>") 15
Ce qu'il faut retenir *16*
Résumé simple 16
_serve_static(self, path) *17*
Pourquoi ce nom est important 17
Code 17
Logigramme 18

Explication ligne par ligne *19*
filepath = os.path.realpath(os.path.join(STATIC_DIR, path.removeprefix("/static/"))) 19
Pourquoi realpath() est important 19
if not filepath.startswith(STATIC_DIR): 19
self._send_response(403, "<h1>403 - Interdit</h1>") 19
return 19
try: 19
with open(filepath, "rb") as file: 19
content = file.read() 19
ext = path.split(".")[-1] 20
content_type = CONTENT_TYPES.get(ext, "text/plain") 20
self.send_response(200) 20
self.send_header("Content-Type", content_type) 20
self.end_headers() 20
self.wfile.write(content) 20
except FileNotFoundError: 20
self._send_response(404, "<h1>404 - Fichier non trouvé</h1>") 20
Ce qu'il faut retenir *21*
Résumé simple 21
_send_response(self, code, html) *22*
Pourquoi ce nom est important 22
Code 22
Explication ligne par ligne *22*
self.send_response(code) 22
self.send_header("Content-Type", "text/html; charset=utf-8") 23
self.end_headers() 23
self.wfile.write(html.encode("utf-8")) 23
Rôle général de la fonction 23
Exemple d'utilisation 24
Ce qu'il faut retenir *24*
Résumé simple 24
*log_message(self, format, *args)* *25*
Pourquoi ce nom est important 25
Rôle de log_message(self, format, *args) 25
Code 25
Explication ligne par ligne *25*
def log_message(self, format, *args): 25
self 25
format 25
*args 26
print(f"[{self.address_string()}] {format % args}") 26
self.address_string() 26
format % args 26
Résultat final affiché 26
Ce qu'il faut retenir *26*
Résumé simple 26
Etape 4 : Créer le launcher *27*
if __name__ == "__main__": *27*
Pourquoi cette écriture est importante 27
Code 27
Diagramme 28
Explication ligne par ligne *29*
server = HTTPServer((APP_HOST, APP_PORT), RequestHandler) 29
print(f"Serveur démarré sur http://{APP_HOST}:{APP_PORT}") 29
try: 29
server.serve_forever() 29
except KeyboardInterrupt: 29
print("\nServeur arrêté.") 29
server.server_close() 29
Ce qu'il faut retenir *30*
Résumé simple 30
ANALYSE D'APP.PY PAR CLAUDE.IA 31
POINTS DE QUALITE VALIDES 31

Plan de développement de app.py

Étape 1 : Définir la mission du fichier

Le fichier app.py est le point d'entrée principal de l'application web.
C'est lui qui lance le serveur HTTP et qui organise la circulation des requêtes entre le navigateur et l'application.

Rôle général de app.py

Le fichier app.py sert à :

- Démarrer le serveur web Python.
- Écouter les requêtes envoyées par le navigateur.
- Analyser l'URL demandée.
- Rechercher la route correspondante.
- Appeler la bonne fonction de traitement.
- Renvoyer une réponse HTTP au navigateur.
- Servir les fichiers statiques nécessaires à l'interface, comme le CSS ou le JavaScript.

Remarque importante

app.py ne contient pas la logique métier complète de l'application.

Son rôle principal est de :

- Recevoir.
- Analyser.
- Router.
- Déléguer.
- Répondre.

Étape 2 : Importer les objets et les constantes globales utiles

En Python, un import permet de charger du code externe pour l'utiliser dans un fichier.
Sans imports, on ne peut utiliser que les fonctions de base du langage.

Code étudié

```python
import os
from http.server import HTTPServer, BaseHTTPRequestHandler
from urllib.parse import urlparse
from config import APP_HOST, APP_PORT
from routes import ROUTES

STATIC_DIR    = os.path.join(os.path.dirname(__file__), "static")
CONTENT_TYPES = {"css": "text/css", "js": "application/javascript"}
```

import os

os est un module de la librairie standard Python (fournie avec Python, rien à installer).
Il donne accès aux fonctions du système d'exploitation : lire des chemins de fichiers, naviguer dans les dossiers, etc.

from http.server import HTTPServer, BaseHTTPRequestHandler

Cet import vient de la bibliothèque standard Python, il sert à créer un petit serveur web sans framework.

HTTPServer

C'est l'objet serveur lui-même, son rôle :

- Écouter sur une adresse @IP et un port.
- Attendre les requêtes du navigateur.
- Transmettre chaque requête à une classe qui sait la traiter.

Exemple d'usage : server = HTTPServer(("127.0.0.1", 8000), RequestHandler)

Ici :

- **127.0.0.1** = adresse locale ;
- **8000** = port ;
- **RequestHandler** = classe qui va traiter les requêtes.

Donc HTTPServer = la machine qui écoute.

BaseHTTPRequestHandler

C'est la classe de base fournie par Python pour traiter les requêtes HTTP.

Utilisation :

- class RequestHandler(BaseHTTPRequestHandler):
  pass (permet de laisser une classe vide en développement)

Ensuite on ajoute des méthodes pour traiter les requêtes GET et POST comme :

- def do_GET(self):
- def do_POST(self):

Donc BaseHTTPRequestHandler = Classe qui sait recevoir une requête HTTP.

from urllib.parse import urlparse, parse_qs

Cet import sert à analyser l'URL reçue.
Quand le navigateur demande une page, il envoie une URL comme : /clients/edit?id=ALFKI
Il faut la découper proprement.

urlparse

urlparse() sert à découper l'URL en morceaux.

Exemple : parsed = urlparse("/clients/edit?id=ALFKI")

On peut ensuite récupérer :

- parsed.path qui vaut : /clients/edit
- parsed.query qui vaut : id=ALFKI

Donc urlparse() sert à séparer :

- Le chemin ;
- La partie paramètres.

parses

parse_qs() sert à transformer la partie paramètres en dictionnaire.

Exemple : params = parse_qs("id=ALFKI")
Résultat : {'id': ['ALFKI']}

Pourquoi une liste ?
Parce qu'une URL peut contenir plusieurs valeurs pour le même paramètre.

Exemple : ?id=ALFKI&id=BONAP
Donc souvent on récupère la première valeur comme ça : client_id = params.get("id", [None])[0]

from config import APP_HOST, APP_PORT

config est notre propre fichier config.py.
Cette ligne importe la configuration d'accès au serveur WEB.

Exemple dans config.py :

- APP_HOST = "127.0.0.1"
- APP_PORT = 8000

Ces valeurs servent à lancer le serveur.
Exemple : server = HTTPServer((APP_HOST, APP_PORT), RequestHandler)

C'est utile car au lieu d'écrire en dur : server = HTTPServer(("127.0.0.1", 8000), RequestHandler), on centralise les paramètres dans config.py.

Avantages :

- Plus propre.
- Plus modifiable.
- Plus réutilisable.
- Plus cohérent avec une architecture MVC.

from routes import ROUTES

ROUTE est notre propre fichier route.py.
Cette ligne importe la table de routage de l'application.

ROUTES contient la liste ou le dictionnaire des routes disponibles.
Son rôle est de :

- Centraliser les routes.
- Éviter de les coder en dur partout dans app.py.
- Rendre le routage plus propre et plus facile à maintenir.

Exemple de rôle : associer une URL à une fonction de traitement.

STATIC_DIR = os.path.join(os.path.dirname(__file__), "static")

Cette constante contient le chemin du dossier static.
Elle sert à dire au programme où se trouvent les fichiers statiques de l'application.

Détail :

- __file__ = chemin du fichier Python courant.
- os.path.dirname(__file__) = dossier dans lequel se trouve app.py.
- os.path.join(..., "static") = ajoute le sous-dossier static.

Donc STATIC_DIR représente le chemin réel du dossier contenant les fichiers comme :

- style.css
- app.js

CONTENT_TYPES = {"css": "text/css", "js": "application/javascript"}

Cette constante associe chaque extension de fichier à son type MIME.
Elle sert quand l'application envoie un fichier statique au navigateur.

Exemples

- css : text/css
- js : application/javascript

Le navigateur a besoin de cette information pour savoir comment traiter le fichier reçu.

Résumé de l'étape

Le début de app.py prépare :

- Les outils système avec os.
- Le serveur HTTP avec HTTPServer.
- Le gestionnaire de requêtes avec BaseHTTPRequestHandler.
- L'analyse d'URL avec urlparse.
- La configuration du serveur avec APP_HOST et APP_PORT.
- Le routage avec ROUTES.
- L'accès aux fichiers statiques avec STATIC_DIR.
- Le type des fichiers envoyés avec CONTENT_TYPES.

Ce qu'il faut retenir

Avant même de coder les fonctions, app.py a besoin de :

- Savoir écouter.
- Savoir analyser.
- Savoir router.
- Savoir retrouver les fichiers statiques.

Étape 3 : Créer la classe RequestHandler

RequestHandler est la classe qui reçoit et traite chaque requête HTTP envoyée par le navigateur.

Rôle de la classe RequestHandler

RequestHandler est la classe chargée de :

- Recevoir les requêtes envoyées par le navigateur.
- Identifier leur type (GET, POST, etc.).
- Analyser l'URL demandée.
- Appeler la bonne fonction de traitement.
- Envoyer une réponse au navigateur.

Autrement dit, c'est elle qui fait le lien entre :

- Le serveur HTTP ;
- Les routes définies dans l'application ;
- Les contrôleurs ;
- Et la réponse finale envoyée au client.

Les fonctions principales de RequestHandler :

- do_GET(self) : pour afficher des pages.
- do_POST(self) : pour traiter les formulaires.
- _serve_static(self, path) : pour envoyer les fichiers CSS/JS.
- _send_response(self, code, html) : pour envoyer une réponse HTML.
- log_message(self, format, *args) : pour afficher les logs dans le terminal.

do_GET(self)

Cette fonction sera appelée automatiquement quand une requête **GET** arrive.

Une requête GET, c'est ce que le navigateur envoie quand on :

- Tape une URL.
- Clique sur un lien.
- Recharge une page.
- Demande un fichier CSS.

Exemples :

- /clients
- /clients/add
- /clients/edit?id=ALFKI

Important
Le nom do_GET est imposé par BaseHTTPRequestHandler.

Donc :

- do_GET = obligatoire pour les requêtes **GET**
- self = l'objet qui traite la requête en cours

Déclaration de la classe

class RequestHandler(BaseHTTPRequestHandler):

Pourquoi cette écriture est importante

RequestHandler

C'est le nom choisi pour la classe.
Ce nom est libre : on pourrait l'appeler autrement.
Mais ici, RequestHandler est un bon nom, car il indique clairement que la classe traite les requêtes.

BaseHTTPRequestHandler

Cette classe vient du module http.server.
Elle fournit les mécanismes de base pour recevoir des requêtes HTTP et permet ensuite de redéfinir des méthodes comme :

- do_GET(self)
- do_POST(self)

Grâce à cet héritage, Python saura automatiquement quelle fonction appeler selon le type de requête reçue.

Ce que cette classe va contenir

La classe RequestHandler contiendra plusieurs fonctions importantes :

- do_GET(self) : pour traiter les requêtes GET ;
- do_POST(self) : pour traiter les requêtes POST ;
- _serve_static(self, path) : pour envoyer les fichiers statiques ;
- _send_response(self, code, html) : pour envoyer une réponse HTML ;
- log_message(self, format, *args) : pour personnaliser les logs du terminal.

Ce qu'il faut retenir

La classe RequestHandler est le gestionnaire central des requêtes HTTP.
C'est elle qui reçoit la demande du navigateur et décide :

- Comment la lire.
- Où l'envoyer.
- Et quoi répondre.

Résumé simple

On peut résumer ainsi :

- HTTPServer écoute.
- RequestHandler traite.
- Les contrôleurs exécutent.
- Le navigateur reçoit la réponse.

do_POST(self)

Cette fonction sera appelée automatiquement quand une requête **POST** arrive.

Rôle de do_GET(self)

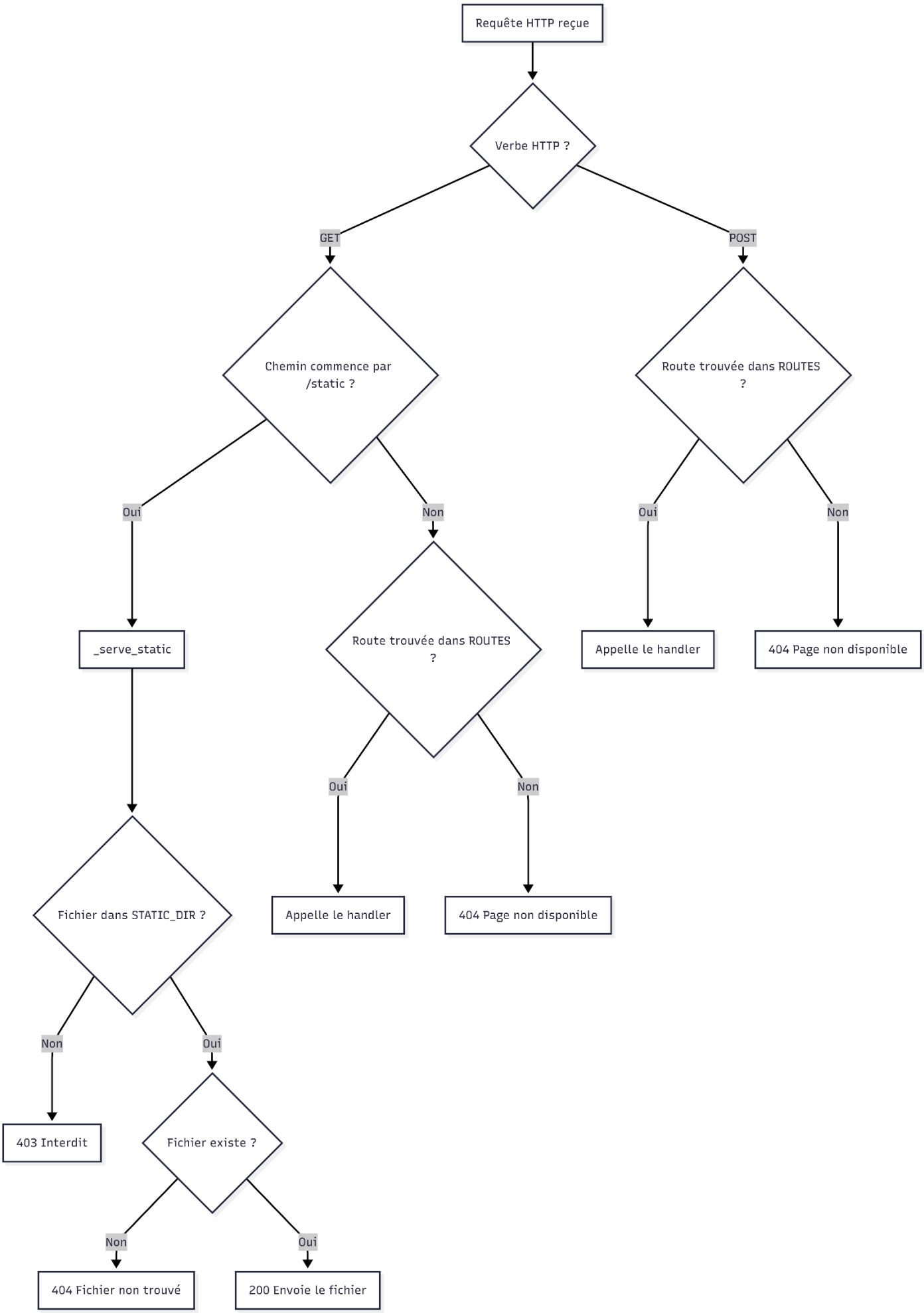
Cette fonction sert à :

- Lire l'URL demandée.
- Extraire le chemin.
- Rechercher la route correspondante dans ROUTES.
- Appeler la bonne fonction de traitement.
- Envoyer une erreur si la route n'existe pas.
- Gérer à part les fichiers statiques.

Code

```
def do_GET(self):
    parsed = urlparse(self.path)
    path = parsed.path

    if path.startswith("/static/"):
        self._serve_static(path)
        return

    route = ROUTES.get(("GET", path))

    if route is not None:
        route(self)
    else:
        self._send_response(404, "<h1>404 - Page non disponible</h1>")
```

Logigramme


Requête HTTP reçue
Verbe HTTP ?
GET
POST
Chemin commence par /static ?
Route trouvée dans ROUTES ?
Oui
Non
Oui
Non
_serve_static
Route trouvée dans ROUTES ?
Appelle le handler
404 Page non disponible
Oui
Non
Appelle le handler
404 Page non disponible
Fichier dans STATIC_DIR ?
Non
Oui
403 Interdit
Fichier existe ?
Non
Oui
404 Fichier non trouvé
200 Envoie le fichier

Explication ligne par ligne

parsed = urlparse(self.path)

Cette ligne analyse l'URL reçue.

Exemple :

- /clients/edit?id=ALFKI
- self.path contient toute cette chaîne.
  urlparse() permet de la découper proprement.

path = parsed.path

Cette ligne récupère uniquement le chemin de l'URL.
Avec :

- /clients/edit?id=ALFKI
- on obtient : /clients/edit

C'est ce chemin qui sert ensuite à chercher la bonne route.

if path.startswith("/static/"):

Cette condition vérifie si l'URL demandée correspond à un fichier statique.

Exemples :

- /static/style.css
- /static/app.js

Les fichiers statiques ne passent pas par le système de routes classiques.
Ils sont envoyés directement par une fonction spécialisée.

self._serve_static(path)

Si la requête vise un fichier statique, cette ligne appelle la fonction chargée de lire le fichier sur le disque et de l'envoyer au navigateur.

return

Cette ligne arrête la fonction immédiatement après l'envoi du fichier statique.
C'est important pour éviter que le programme continue ensuite à chercher une route inutilement.

route = ROUTES.get(("GET", path))

Cette ligne cherche dans la table ROUTES si une route correspond à :

- la méthode HTTP GET
- et le chemin demandé

Exemple : ("GET", "/clients")
Si cette clé existe dans ROUTES, on récupère la fonction associée.

if route is not None:

Cette condition vérifie si une route a bien été trouvée.
Si oui, cela signifie que l'URL demandée est connue par l'application.

route(self)

Cette ligne appelle la fonction associée à la route trouvée.
Elle transmet self, c'est-à-dire l'objet qui gère la requête en cours.

C'est cette fonction qui se chargera ensuite du traitement réel :

- Affichage d'une page.
- Appel d'un contrôleur.
- Récupération éventuelle des paramètres.

else:

Si aucune route ne correspond, on passe ici.
Cela signifie que l'utilisateur a demandé une page qui n'existe pas dans l'application.

self._send_response(404, "<h1>404 - Page non disponible</h1>")

Cette ligne envoie une erreur HTTP 404 au navigateur.
Le code 404 signifie : **ressource non trouvée**.

Ce qu'il faut retenir

do_GET(self) sert à :

- Traiter toutes les requêtes GET.
- Distinguer les fichiers statiques des routes normales.
- Rechercher la bonne route dans ROUTES.
- Appeler la fonction associée.
- Envoyer une erreur 404 si rien ne correspond.

Résumé simple

On peut résumer le travail de do_GET(self) ainsi :

- Analyser l'URL ;
- Récupérer le chemin ;
- Si c'est un fichier statique, l'envoyer ;
- Sinon, chercher la route ;
- Appeler la bonne fonction ;
- Ou envoyer une 404.

do_POST(self)

Cette fonction est appelée automatiquement quand une requête HTTP de type POST arrive.
Une requête POST, c'est ce que le navigateur envoie lorsqu'un utilisateur soumet un formulaire.

Exemples :

- Enregistrement d'un nouveau client.
- Validation de la modification d'un client.

Pourquoi ce nom est important

Le nom do_POST est imposé par BaseHTTPRequestHandler.

Si on écrit par exemple :

- def ma_fonction_post(self):
  ou :
- def do_Post(self):

La fonction ne sera jamais appelée automatiquement.

Donc :

- do_POST = obligatoire pour les requêtes POST ;
- self = l'objet qui traite la requête en cours.

Rôle de do_POST(self)

Cette fonction sert à :

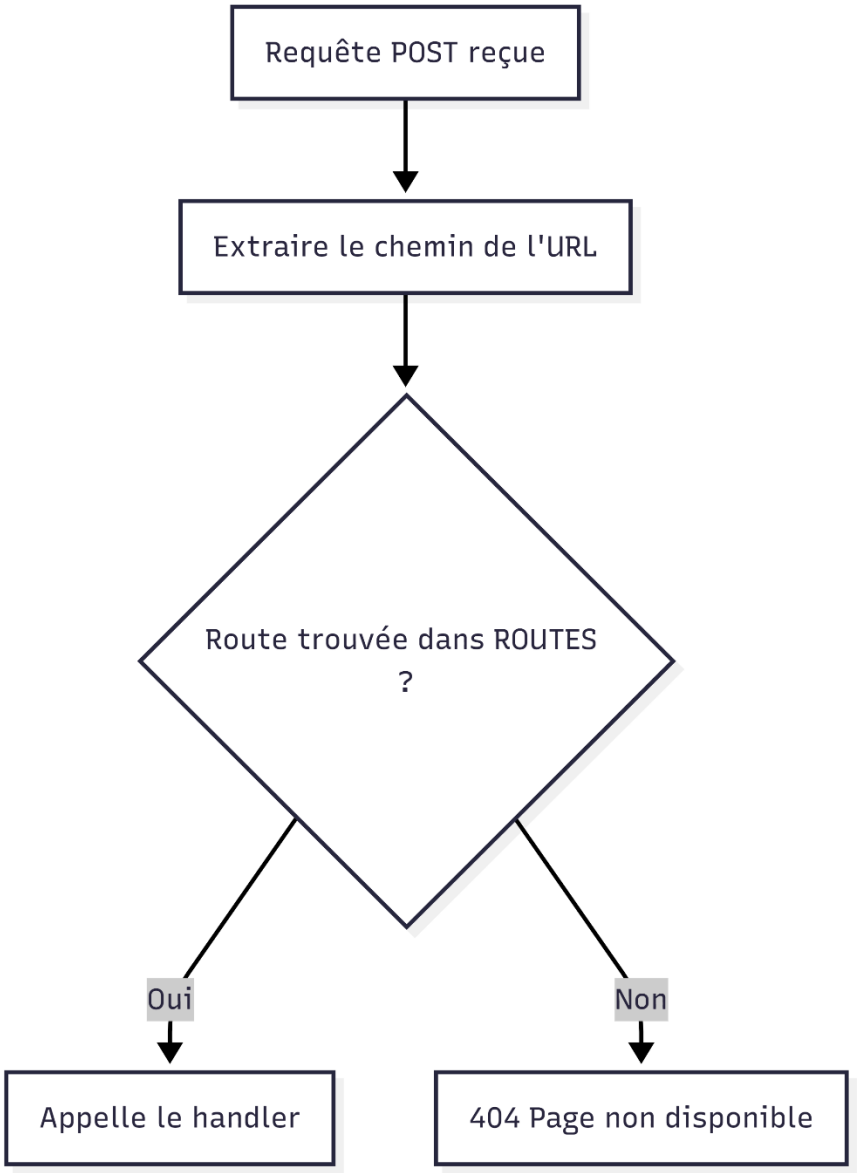
- Lire l'URL demandée.
- Rechercher la route correspondante dans ROUTES.
- Appeler la bonne fonction de traitement.
- Envoyer une erreur si la route n'existe pas.

Contrairement à do_GET(self), elle ne sert pas à afficher des pages directement.
Elle sert surtout à traiter les données envoyées par le navigateur.

Code

```
def do_POST(self):
    parsed = urlparse(self.path)
    path = parsed.path

    route = ROUTES.get(("POST", path))

    if route is not None:
        route(self)
    else:
        self._send_response(404, "<h1>404 - Page non disponible</h1>")
```

Logigramme


Requête POST reçue
Extraire le chemin de l'URL
Route trouvée dans ROUTES ?
Oui
Non
Appelle le handler
404 Page non disponible

Explication ligne par ligne

parsed = urlparse(self.path)

Cette ligne analyse l'URL reçue.
Même dans une requête POST, l'URL contient encore un chemin, par exemple :

- /clients/add
  ou :
- /clients/edit

urlparse() permet de découper proprement cette URL.

path = parsed.path

Cette ligne récupère uniquement le chemin.
Exemple :

- Si self.path vaut /clients/add
- Alors path vaut /clients/add

C'est ce chemin qui permettra de chercher la bonne route.

route = ROUTES.get(("POST", path))

Cette ligne cherche dans ROUTES une route correspondant à :

- La méthode HTTP POST
- Et au chemin demandé

Exemple : ("POST", "/clients/add")
Si cette route existe, on récupère la fonction associée.

if route is not None:

Cette condition vérifie si une route a bien été trouvée.
Si oui, cela signifie que l'application sait comment traiter cette requête POST.

route(self)

Cette ligne appelle la fonction associée à la route.
Elle transmet self, c'est-à-dire l'objet qui gère la requête actuelle.

La fonction appelée se chargera ensuite de :

- Lire les données du formulaire.
- Appeler le contrôleur.
- Lancer un ajout ou une modification.
- Renvoyer une réponse au navigateur.

else:

Si aucune route POST ne correspond au chemin demandé, on arrive ici.
Cela signifie que l'utilisateur a envoyé une requête vers une URL non prévue par l'application.

self._send_response(404, "<h1>404 - Page non disponible</h1>")

Cette ligne envoie une erreur HTTP 404.
Le code 404 signifie que la ressource demandée n'existe pas.

Ce qu'il faut retenir

do_POST(self) sert à :

- Traiter les requêtes POST.
- Analyser le chemin demandé.
- Rechercher la route correspondante dans ROUTES.
- Appeler la bonne fonction de traitement.
- Envoyer une erreur 404 si la route n'existe pas.

Résumé simple

On peut résumer le travail de do_POST(self) ainsi :

- Lire l'URL.
- Récupérer le chemin.
- Chercher une route POST correspondante.
- Appeler la bonne fonction.
- Ou envoyer une erreur 404.

La différence principale avec do_GET(self) est la suivante :

- do_GET(self) sert surtout à afficher ;
- do_POST(self) sert surtout à traiter les données envoyées.

_serve_static(self, path)

Cette fonction sert à envoyer au navigateur les fichiers statiques de l'application, par exemple :

- Une feuille de style CSS.
- Un fichier JavaScript.

Exemples de fichiers demandés :

- /static/style.css
- /static/app.js

Son rôle est de :

- Retrouver le fichier demandé dans le dossier static.
- Vérifier que ce fichier est bien autorisé.
- Lire son contenu.
- L'envoyer au navigateur avec le bon type MIME.
- Renvoyer une erreur si le fichier n'existe pas ou si l'accès est interdit.

Pourquoi ce nom est important

Le nom _serve_static n'est pas imposé par Python comme do_GET ou do_POST.
C'est un nom choisi librement par le programmeur pour rendre le code plus clair.
Le préfixe « _ » signifie simplement que c'est une méthode interne à la classe, utilisée par le programme lui-même.

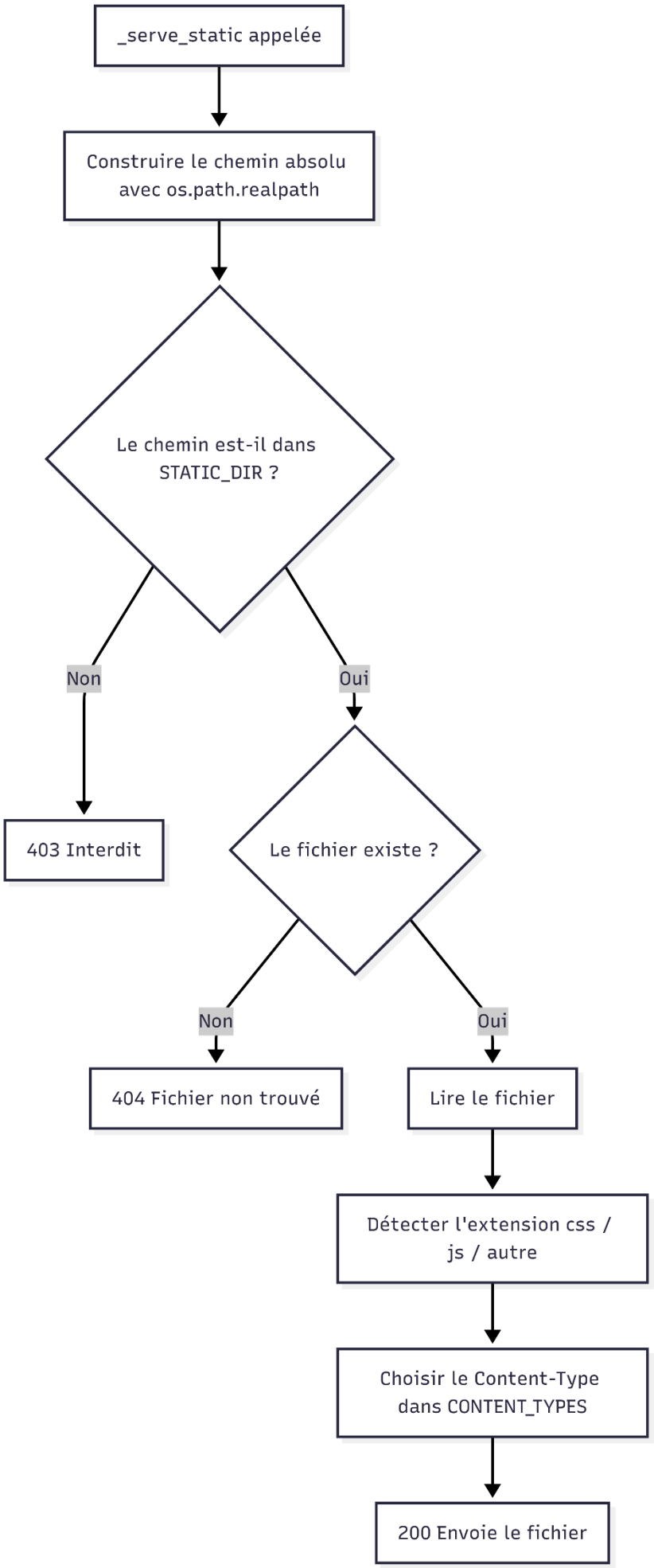
Donc :

- _serve_static = nom libre, choisi pour la lisibilité.
- self = l'objet qui traite la requête en cours.
- path = le chemin demandé par le navigateur.

Code

```python
def _serve_static(self, path):
    filepath = os.path.realpath(
        os.path.join(STATIC_DIR, path.removeprefix("/static/"))
    )

    if not filepath.startswith(STATIC_DIR):
        self._send_response(403, "<h1>403 - Interdit</h1>")
        return

    try:
        with open(filepath, "rb") as file:
            content = file.read()

        ext = path.split(".")[-1]
        content_type = CONTENT_TYPES.get(ext, "text/plain")

        self.send_response(200)
        self.send_header("Content-Type", content_type)
        self.end_headers()
        self.wfile.write(content)

    except FileNotFoundError:
        self._send_response(404, "<h1>404 - Fichier non trouvé</h1>")
```

Logigramme


_serve_static appelée
Construire le chemin absolu avec os.path.realpath
Le chemin est-il dans STATIC_DIR ?
Non
Oui
403 Interdit
Le fichier existe ?
Non
Oui
404 Fichier non trouvé
Lire le fichier
Détecter l'extension css / js / autre
Choisir le Content-Type dans CONTENT_TYPES
200 Envoie le fichier

Explication ligne par ligne

filepath = os.path.realpath(os.path.join(STATIC_DIR, path.removeprefix("/static/")))

Cette ligne construit le chemin réel du fichier demandé.

Si l'URL vaut : /static/style.css
Alors on enlève le préfixe /static/, ce qui donne : style.css
Puis on l'ajoute au dossier STATIC_DIR.
Enfin, os.path.realpath(...) calcule le vrai chemin absolu du fichier.

Pourquoi realpath() est important

C'est une mesure de sécurité, sans cela, un attaquant pourrait essayer une URL comme : /static/../../config.py
Le but serait de sortir du dossier static pour lire un fichier sensible.
realpath() résout les « ../ » et permet ensuite de vérifier si le fichier demandé reste bien dans le dossier autorisé.

if not filepath.startswith(STATIC_DIR):

Cette condition vérifie que le chemin final appartient bien au dossier static.
Si ce n'est pas le cas, cela signifie que la requête tente d'accéder à un fichier en dehors du dossier autorisé.

self._send_response(403, "<h1>403 - Interdit</h1>")

Si l'accès est interdit, cette ligne envoie une réponse HTTP **403**.
Le code 403 signifie.

- Accès refusé.
- Le serveur a compris la requête.
- Mais il ne l'autorise pas.

return

Cette ligne arrête immédiatement la fonction.
Elle évite que le programme continue à lire ou envoyer un fichier après avoir déjà refusé l'accès.

try:

Ce bloc permet de tenter l'ouverture du fichier demandé.
Il sert à gérer proprement le cas où le fichier n'existe pas.

with open(filepath, "rb") as file:

Cette ligne ouvre le fichier en mode binaire.

Le mode "rb" signifie :

- r = lecture ;
- b = binaire.

Ce mode convient bien à l'envoi d'un fichier au navigateur.

content = file.read()

Cette ligne lit tout le contenu du fichier.
Le contenu est ensuite stocké en mémoire pour être envoyé au navigateur.

ext = path.split(".")[-1]

Cette ligne récupère l'extension du fichier demandé.

Exemples :
- style.css : css
- app.js : js

Cela permet de savoir quel type de contenu doit être envoyé.

content_type = CONTENT_TYPES.get(ext, "text/plain")

Cette ligne cherche le type MIME correspondant à l'extension du fichier.

Exemples :
- css : text/css
- js : application/javascript

Si l'extension n'est pas connue, on utilise text/plain par défaut.

self.send_response(200)

Cette ligne envoie le code HTTP 200.
Le code 200 signifie que tout s'est bien passé.

self.send_header("Content-Type", content_type)

Cette ligne envoie l'en-tête HTTP Content-Type.
Elle indique au navigateur le type du fichier qu'il reçoit.

Exemples :
- text/css
- application/javascript

self.end_headers()

Cette ligne indique la fin des en-têtes HTTP.
Après cela, le programme peut envoyer le contenu du fichier.

self.wfile.write(content)

Cette ligne envoie le contenu du fichier au navigateur.
C'est l'écriture réelle de la réponse HTTP.

except FileNotFoundError:

Cette partie gère le cas où le fichier n'existe pas sur le disque.

Exemple :
- Le navigateur demande /static/style.css
- Mais le fichier style.css est absent du dossier static

Dans ce cas, Python lève une erreur FileNotFoundError.

self._send_response(404, "<h1>404 - Fichier non trouvé</h1>")

Si le fichier est introuvable, cette ligne envoie une réponse HTTP 404.
Le code 404 signifie : ressource non trouvée.

Ce qu’il faut retenir

_serve_static(self, path) sert à :

- Traiter les demandes de fichiers statiques.
- Reconstruire le chemin réel du fichier.
- Empêcher les accès hors du dossier static.
- Lire le fichier.
- Envoyer son contenu avec le bon type MIME.
- Renvoyer 403 si l’accès est interdit.
- Renvoyer 404 si le fichier n’existe pas.

Résumé simple

On peut résumer le travail de _serve_static(self, path) ainsi :

- Recevoir un chemin de fichier.
- Vérifier qu’il est autorisé.
- Ouvrir le fichier.
- Déterminer son type.
- L’envoyer au navigateur.
- Ou renvoyer une erreur si besoin.

_send_response(self, code, html)

Cette fonction est une méthode utilitaire.

Elle sert à envoyer facilement une réponse HTTP au navigateur avec :

- Un code HTTP.
- Un contenu HTML.

Exemples d'utilisation :

- Envoyer une page normale avec le code 200.
- Envoyer une page d'erreur avec le code 404.
- Envoyer une page d'accès interdit avec le code 403.

Pourquoi ce nom est important

Le nom _send_response n'est pas imposé par Python.
C'est un nom choisi librement par le programmeur pour rendre le code plus clair.
Le préfixe « _ » indique qu'il s'agit d'une méthode interne à la classe RequestHandler, utilisée par le programme lui-même.

Donc :

- _send_response = nom libre, choisi pour la lisibilité ;
- self = l'objet qui traite la requête en cours ;
- code = le code HTTP à envoyer ;
- html = le contenu HTML à envoyer au navigateur.

Code

```
def _send_response(self, code, html):
    self.send_response(code)
    self.send_header("Content-Type", "text/html; charset=utf-8")
    self.end_headers()
    self.wfile.write(html.encode("utf-8"))
```

Explication ligne par ligne

self.send_response(code)

Cette ligne envoie le code HTTP de la réponse.

Exemples :

- 200 → la requête a réussi.
- 404 → la page demandée n'existe pas.
- 403 → l'accès est interdit.

Le navigateur lit ce code pour savoir si la requête a réussi ou échoué.

self.send_header("Content-Type", "text/html; charset=utf-8")

Cette ligne envoie un en-tête HTTP.

Ici, elle précise que le contenu renvoyé est :

- Du **HTML**.
- Encodé en **UTF-8**.

Pourquoi charset=utf-8 est important ?

Cela permet d'afficher correctement :

- Les accents.
- Les caractères spéciaux.
- Le texte français.

Sans cela, certains caractères pourraient mal s'afficher dans le navigateur.

self.end_headers()

Cette ligne indique que la partie "en-têtes HTTP" est terminée.
Après cette ligne, on peut envoyer le contenu réel de la réponse.

self.wfile.write(html.encode("utf-8"))

Cette ligne envoie le contenu HTML au navigateur.

Pourquoi encode("utf-8") ?

Le navigateur attend des octets, pas une simple chaîne Python.
Donc :

- html est une chaîne de caractères ;
- encode("utf-8") la transforme en données binaires ;
- wfile.write(...) envoie ces données dans la réponse HTTP.

Rôle général de la fonction

Cette fonction évite de répéter partout les mêmes lignes :

- send_response(...)
- send_header(...)
- end_headers()
- wfile.write(...)

Au lieu de recopier cela dans chaque fonction, on centralise l'envoi des réponses dans une seule méthode.
C'est plus :

- Propre ;
- Lisible ;
- Réutilisable.

Exemple d'utilisation

Exemple 1 : réponse normale

```
self._send_response(200, "<h1>Liste des clients</h1>")
```

Le navigateur reçoit une erreur 404.

Exemple 2 : page non trouvée

```
self._send_response(404, "<h1>404 - Page non disponible</h1>")
```

Le navigateur reçoit une erreur 404.

Exemple 3 : accès interdit

```
self._send_response(403, "<h1>403 - Interdit</h1>")
```

Le navigateur reçoit une erreur 403.

Ce qu'il faut retenir

_send_response(self, code, html) sert à :

- Envoyer une réponse HTTP simple.
- Préciser le code http.
- Envoyer du HTML encodé en UTF-8.
- Éviter de répéter le même code dans plusieurs endroits.

Résumé simple

On peut résumer son rôle ainsi :

- Fixer le statut de la réponse.
- Annoncer que le contenu est du HTML.
- Terminer les en-têtes.
- Envoyer la page au navigateur.

log_message(self, format, *args)

Cette fonction sert à personnaliser l'affichage des logs dans le terminal.

Un log est un message affiché par le serveur pour indiquer qu'une requête HTTP a été reçue et traitée.

Exemple d'informations affichées :

- L'adresse IP du client.
- La route demandée.
- La méthode HTTP utilisée.
- Le code de réponse envoyé.

Pourquoi ce nom est important

Le nom log_message existe déjà dans BaseHTTPRequestHandler.

Cela signifie que cette méthode est fournie par la classe de base et qu'il est possible de la redéfinir pour changer la façon dont les messages apparaissent dans le terminal.

Donc :

- log_message = méthode reconnue par BaseHTTPRequestHandler ;
- En la redéfinissant, on personnalise les logs du serveur.

Rôle de log_message(self, format, *args)

Cette fonction sert à :

- Remplacer l'affichage par défaut du serveur HTTP.
- Rendre les logs plus lisibles.
- Faciliter le suivi des requêtes pendant les tests.

Elle n'est pas indispensable au fonctionnement de l'application, mais elle est très utile pour le debug.

Code

```
def log_message(self, format, *args):
    """
    Surcharge du logger par défaut de BaseHTTPRequestHandler.
    Affiche chaque requête reçue dans le terminal avec l'adresse IP du client.
    """
    print(f"[{self.address_string()}] {format % args}")
```

Explication ligne par ligne

def log_message(self, format, *args):

Cette ligne définit la fonction qui remplace la version par défaut.

self

Représente l'objet qui traite la requête en cours.

format

Contient le modèle du message à afficher.

*args

Contient les valeurs à insérer dans ce modèle.
Le symbole * signifie qu'il peut y avoir plusieurs arguments.

print(f"[{self.address_string()}] {format % args}")

Cette ligne affiche le message dans le terminal.

self.address_string()

Retourne l'adresse IP du client ayant envoyé la requête.
Exemple : 127.0.0.1

format % args

Construit le message final du log à partir du modèle fourni par BaseHTTPRequestHandler.
Exemple de message généré : "GET /clients HTTP/1.1" 200 –

Résultat final affiché

On obtient un affichage du type : [127.0.0.1] "GET /clients HTTP/1.1" 200 -

Ce qu'il faut retenir

log_message(self, format, *args) sert à :

- Personnaliser les logs du serveur.
- Afficher l'adresse IP du client.
- Rendre les messages du terminal plus lisibles.
- Aider au debug pendant le développement.

Résumé simple

Cette fonction ne sert pas à afficher une page web.
Elle sert uniquement à afficher dans le terminal un message indiquant qu'une requête a été reçue et traitée.

Etape 4 : Créer le launcher

if __name__ == "__main__":

Ce bloc correspond au point d'entrée principal du programme.
Il sert à :

- Créer le serveur http.
- Le démarrer sur l'adresse et le port définis dans config.py.
- Le laisser fonctionner en continu.
- L'arrêter proprement lorsqu'on interrompt le programme.

Pourquoi cette écriture est importante

La condition : if __name__ == "__main__":

Permet de dire :

- Exécuter ce bloc seulement si le fichier app.py est lancé directement.
- Ne pas l'exécuter si le fichier est importé depuis un autre module.

Exemple :

- Si on lance **python3 app.py** : le bloc s'exécute.
- Si un autre fichier fait **import app** : ce bloc ne s'exécute pas.

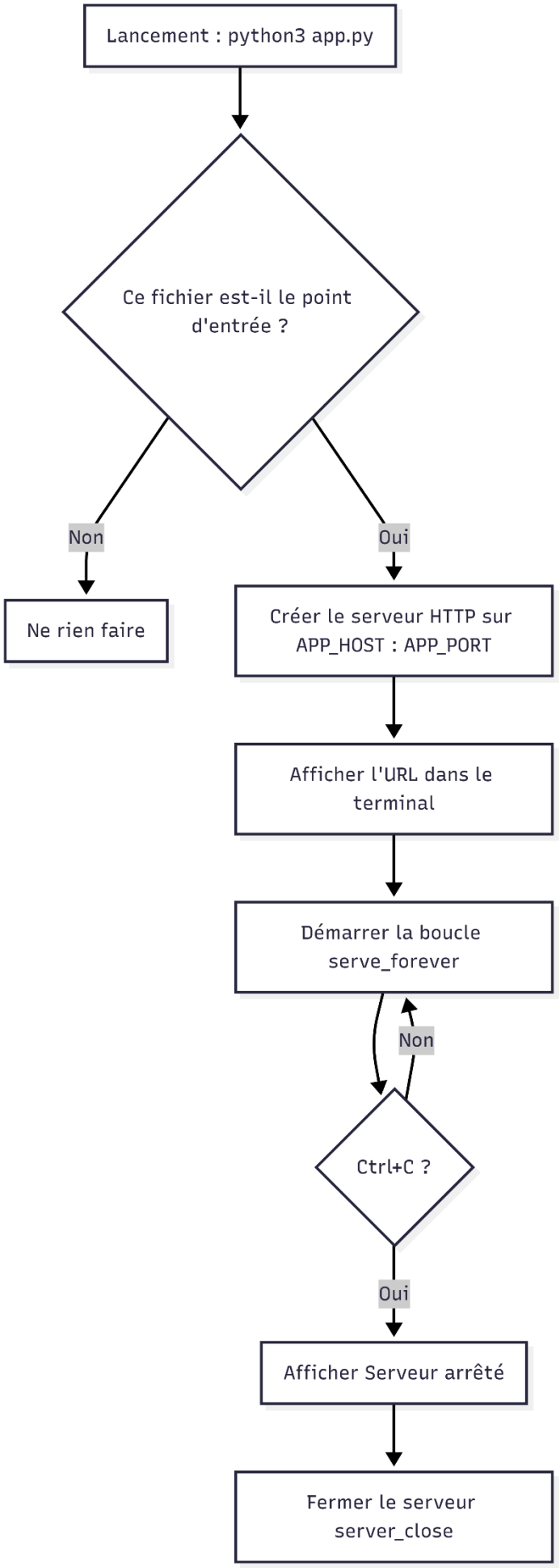
Donc :

- __name__ == "__main__" = vrai si le fichier est le programme principal.
- Ce test évite de démarrer le serveur automatiquement lors d'un import.

Code

```
if __name__ == "__main__":
    server = HTTPServer((APP_HOST, APP_PORT), RequestHandler)
    print(f"Serveur démarré sur http://{APP_HOST}:{APP_PORT}")

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nServeur arrêté.")
        server.server_close()
```

Diagramme


Lancement : python3 app.py
Ce fichier est-il le point d'entrée ?
Non
Oui
Ne rien faire
Créer le serveur HTTP sur APP_HOST : APP_PORT
Afficher l'URL dans le terminal
Démarrer la boucle serve_forever
Non
Ctrl+C ?
Oui
Afficher Serveur arrêté
Fermer le serveur server_close

Explication ligne par ligne

server = HTTPServer((APP_HOST, APP_PORT), RequestHandler)

Cette ligne crée le serveur HTTP.

Elle indique :

- L'adresse d'écoute avec APP_HOST.
- Le port d'écoute avec APP_PORT.
- La classe RequestHandler qui sera utilisée pour traiter chaque requête reçue.

Exemple :

- APP_HOST = "127.0.0.1"
- APP_PORT = 8000

Le serveur écoutera alors sur : http://127.0.0.1:8000

print(f"Serveur démarré sur http://{APP_HOST}:{APP_PORT}")

Cette ligne affiche un message dans le terminal pour indiquer que le serveur a bien démarré.
Elle permet de connaître immédiatement l'adresse à utiliser dans le navigateur.

Exemple d'affichage : Serveur démarré sur http://127.0.0.1:8000

try:

Ce bloc permet de lancer le serveur tout en préparant la gestion d'un arrêt manuel.
Il sert à intercepter proprement l'interruption du programme.

server.serve_forever()

Cette méthode démarre le serveur en continu.

Elle signifie :

- Attendre les requêtes du navigateur.
- Les traiter une par une.
- Rester actif tant qu'aucun arrêt n'est demandé.

Sans cette ligne, le serveur serait créé puis le programme se terminerait immédiatement.

except KeyboardInterrupt:

Cette partie intercepte l'arrêt manuel du programme, en général lorsqu'on appuie sur : Ctrl + C
Cela permet d'éviter un arrêt brutal.

print("\nServeur arrêté.")

Cette ligne affiche un message dans le terminal pour indiquer que le serveur a été interrompu.
L'écriture \n ajoute simplement un retour à la ligne avant le message.

server.server_close()

Cette ligne ferme proprement le serveur.
Elle permet de :

- libérer le port utilisé ;
- arrêter correctement le service ;
- éviter de laisser le serveur dans un état incohérent.

Ce qu’il faut retenir

Ce bloc sert à :

- Lancer l’application seulement si app.py est exécuté directement ;
- Créer le serveur HTTP ;
- Afficher son adresse ;
- Le faire fonctionner en continu ;
- L’arrêter proprement avec Ctrl + C.

Résumé simple

On peut résumer ce bloc ainsi :

- Vérifier que app.py est bien le programme principal ;
- Créer le serveur ;
- Afficher l’adresse de démarrage ;
- Lancer le serveur en continu ;
- Gérer proprement son arrêt.

Analyse d'app.py par claude.ia

Structure générale

Élément	Rôle
Docstring module	Claire, liste les responsabilités
`STATIC_DIR`	Constante calculée une fois au démarrage
`CONTENT_TYPES`	Constante calculée une fois au démarrage
`RequestHandler`	Gestion des requêtes HTTP
`if __name__`	Démarrage propre avec arrêt gracieux

Méthodes

Méthode	Qualité
`do_GET`	Propre, délègue aux bons endroits
`do_POST`	Propre, symétrique à `do_GET`
`_serve_static`	Sécurisée, bien documentée
`_send_response`	Helper simple et réutilisable
`log_message`	Surcharge correcte

Points de qualité validés

- Aucun controller importé : couplage zéro
- Routage entièrement délégué à ROUTES
- Suppressions via POST uniquement
- Docstrings correctement placées
- Imports propres sans inutile
- Constantes au niveau module